Run the LLM orchestrator from your backend

For: the teammate who owns the frontend and the FastAPI backend.

Goal: a button on the frontend starts the orchestrator pipeline (the ML notebooks, then the LLM reports). The backend does not import the orchestrator's code. It starts it as a separate process, the same way it is run in a terminal. The backend therefore depends on one thing only, the folder path. When the orchestrator changes, replace the folder (see step 6); the backend code stays as it is.

0. Before you start: what the orchestrator needs

- **Python 3.11 or newer** for the orchestrator's own venv.
- **The orchestrator's .env** (in the orchestrator folder; copy .env.example and fill it in). Required values: `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY` (the service-role key, server side only, never in the frontend), `INSIGHTS_TABLE=ML_Notebook_Insights`, `REPORTS_TABLE=LLM_Reports`, `GEMINI_API_KEY`, `GROQ_API_KEY` and `LLM_CHAIN` (the model list; the example in .env.example works). Never commit this file.
- **Optional, custom standards:** `CUSTOM_INSIGHTS_TABLE` and `CUSTOM_REPORTS_TABLE` (set both or neither). Leave them out and only the industry standards run.
- **Supabase tables must exist.** `ML_Notebook_Insights` is written by the ML notebooks. `LLM_Reports` is created with `sql/llm_reports.sql` (and `sql/llm_reports_custom.sql` for custom). The run history uses the three tables of `sql/dashboard.sql`; run it once, or set `DASHBOARD_TRACKING=false` to skip the history.
- **The notebooks** run as separate processes and write their insights to Supabase themselves. They receive the same .env automatically, so no extra setup is needed beyond installing their libraries (`pip install -e ".[notebooks]"`, step 1).
- **Internet access** from the server to Supabase, Google (Gemini) and Groq.
- **Check it works before wiring the backend.** In the orchestrator folder with its venv active run `python -m sf_orchestrator.pipeline --dry-run`; it lists the notebooks and runs nothing. Then run `python -m sf_orchestrator.pipeline` once by hand. If that works, the button will work.

1. Put the folder in the project and set it up once

```
<backend root>/
  orchestrator/           <- the whole orchestrator folder, as received
    .env                  <- filled in (Supabase, API keys, tables: copy .env.example and fill it)
    notebooks/
    src/
    ...
  app/                    <- your FastAPI code
```

- Create a virtual environment **inside** the orchestrator folder and install it once: `cd orchestrator`, `python -m venv .venv`, `.venv\Scripts\activate`, `pip install -e ".[notebooks]"` (on Linux or Cloud Foundry use the matching activate command or buildpack install).
- Keep this venv separate from the backend's venv. The notebooks need heavy libraries (pandas, xgboost, shap) that the backend should not carry.
- The orchestrator reads its own `.env` from its own folder. The backend does not need those keys.

2. Add two settings to the backend's .env

```
ORCHESTRATOR_DIR=orchestrator
ORCHESTRATOR_PYTHON=orchestrator/.venv/Scripts/python.exe
```

On Linux or Mac use `orchestrator/.venv/bin/python`.

3. Add one new file: app/orchestrator_runner.py

```
import os
import signal
import subprocess
import sys
import threading
from collections import deque
from datetime import datetime, timezone
from pathlib import Path

ORCH_DIR = Path(os.getenv("ORCHESTRATOR_DIR", "orchestrator")).resolve()
ORCH_PYTHON = os.getenv("ORCHESTRATOR_PYTHON") or sys.executable
STANDARDS = ("all", "industry", "custom")

_lock = threading.Lock()
_job: dict | None = None  # the current or last run


def _kill_tree(proc: subprocess.Popen) -> None:
    if os.name == "nt":
        subprocess.run(["taskkill", "/T", "/F", "/PID", str(proc.pid)], capture_output=True)
    else:
        try:
            os.killpg(proc.pid, signal.SIGKILL)
        except ProcessLookupError:
            pass


def _watch(job: dict, proc: subprocess.Popen) -> None:
    for line in proc.stdout:  # keep the last 300 lines of output
        job["tail"].append(line.rstrip())
    code = proc.wait()
    job["exit_code"] = code
    job["finished_at"] = datetime.now(timezone.utc).isoformat()
    if job["status"] != "stopped":
        job["status"] = "succeeded" if code == 0 else "failed"


def start(standards: str = "all", run_date: str | None = None, force: bool = False) -> dict:
    global _job
    if standards not in STANDARDS:
        raise ValueError(f"standards must be one of {STANDARDS}")
    if run_date:
        datetime.strptime(run_date, "%Y-%m-%d")
    with _lock:
        if _job and _job["status"] == "running":
            raise RuntimeError("A run is already in progress")
        argv = [ORCH_PYTHON, "-u", "-m", "sf_orchestrator.pipeline", "--standards", standards]
        argv += ["--run-date", run_date] if run_date else []
        argv += ["--force"] if force else []
        env = {**os.environ, "PYTHONPATH": str(ORCH_DIR / "src"), "PYTHONUTF8": "1",
               "PYTHONIOENCODING": "utf-8", "SF_TRIGGER": "backend"}
        proc = subprocess.Popen(
            argv, cwd=ORCH_DIR, env=env, stdin=subprocess.DEVNULL,
            stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True, encoding="utf-8", errors="replace",
            **({"creationflags": subprocess.CREATE_NEW_PROCESS_GROUP} if os.name == "nt" else {"start_new_session": True}),
        )
        _job = {"status": "running", "standards": standards, "run_date": run_date, "force": force,
                "started_at": datetime.now(timezone.utc).isoformat(), "finished_at": None,
                "exit_code": None, "tail": deque(maxlen=300), "proc": proc}
        threading.Thread(target=_watch, args=(_job, proc), daemon=True).start()
        return status()


def status(tail: int = 50) -> dict:
    if not _job:
        return {"status": "idle"}
    return {**{k: v for k, v in _job.items() if k not in ("proc", "tail")}, "output": list(_job["tail"])[-tail:]}


def stop() -> dict:
    with _lock:
        if not _job or _job["status"] != "running":
            raise RuntimeError("No run in progress")
        _job["status"] = "stopped"
        _kill_tree(_job["proc"])
    return status()
```

4. Add one router: app/routes_pipeline.py, and register it

```
from fastapi import APIRouter, HTTPException
```

```
from pydantic import BaseModel
from . import orchestrator_runner as runner

router = APIRouter(prefix="/api/pipeline", tags=["pipeline"])


class RunRequest(BaseModel):
    standards: str = "all"          # "all" | "industry" | "custom"
    run_date: str | None = None     # "YYYY-MM-DD", default today (UTC)
    force: bool = False             # regenerate even if reports are current


@router.post("/run", status_code=202)
def run(req: RunRequest):
    try:
        return runner.start(req.standards, req.run_date, req.force)
    except RuntimeError as e:
        raise HTTPException(409, str(e))
    except ValueError as e:
        raise HTTPException(422, str(e))


@router.get("/status")
def status():
    return runner.status()


@router.post("/stop")
def stop():
    try:
        return runner.stop()
    except RuntimeError as e:
        raise HTTPException(409, str(e))
```

In `main.py` add:

```
from app.routes_pipeline import router as pipeline_router
app.include_router(pipeline_router)
```

5. Frontend: the button

```
async function runPipeline() {
  const r = await fetch("/api/pipeline/run", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ standards: "all" }),   // add run_date / force if needed
  });
  if (r.status === 409) return alert("A run is already in progress");
  poll();
}

async function poll() {
  const s = await (await fetch("/api/pipeline/status")).json();
  // show s.status ("running" | "succeeded" | "failed" | "stopped") and s.output (last lines)
  if (s.status === "running") setTimeout(poll, 3000);
  else refreshReports();   // reload the reports from Supabase
}
```

Disable the button while the status is `running`. When it ends with `succeeded`, re-read the `LLM_Reports` table, as you already do.

6. Updating the orchestrator later

Replace the `orchestrator/` folder with the new one, but **keep two things from the old folder: `.env` and `.venv`**. If the new version adds libraries, run `pip install -e ".[notebooks]"` again inside that venv. The backend files stay unchanged.

Things to know

- A full run takes several minutes (notebooks first, then the model calls). That is why it runs in the background and the page polls the status.
- Only one run at a time. A second click returns 409 "already in progress".

- The status is held in the backend's memory. It resets if the backend restarts, and with several workers (`uvicorn --workers 4`) each worker has its own copy. Run one worker, or read the status from the Supabase table `sf_dashboard_runs` (every run is recorded there, with trigger `backend`).
- Exit code 0 = ok, 1 = a notebook or report failed, 2 = configuration error. The `output` lines in the status show the reason.
- On SAP BTP / Cloud Foundry set the orchestrator's variables as app environment variables instead of shipping a .env, and make sure the app has enough memory and a long enough timeout for the notebooks.
- Quick test without the frontend: run `python -m sf_orchestrator.pipeline --dry-run` inside the orchestrator folder (with its venv active). It lists the notebooks and runs nothing.